Singleton Classes Revealed

When you ask an object for its class, Ruby doesn't always tell you the whole truth. Instead of the class that you see, an object can have its own special, hidden class. That's called the *singleton class* of the object. (You can also hear it called the *metaclass* or the *eigenclass*. However, "singleton class" is the official name.)

Methods like Object#class keep the singleton class carefully hidden, but you can work around them. Ruby has a special syntax, based on the class keyword, that places you in the scope of the singleton class:

```
class << an_object
  # your code here
end
```

If you want to get a reference to the singleton class, you can return self out of the scope:

```
obj = Object.new

singleton_class = class << obj
  self
end

singleton_class.class   # => Class
```

That sneaky singleton class was trying to hide, but we managed to find it.

Back in Ruby's old days, you had to return self like we just did to get a reference to the singleton class. These days you can also get a reference to the singleton class with the handy Object#singleton_class method:

```
"abc".singleton_class    # => #<Class:#<String:0x331df0>>
```

The previous example also shows that a singleton class is a class—but a very special one. For starters, it's invisible until you resort to either Object#singleton_class, or the exotic class << syntax. Also, singleton classes have only a single instance (that's where their name comes from), and they can't be inherited. More important, *a singleton class is where an object's Singleton Methods live*:

```
def obj.my_singleton_method; end
singleton_class.instance_methods.grep(/my_/)  # => [:my_singleton_method]
```

To fully understand the consequences of this last point, you have to look deeper into Ruby's object model.